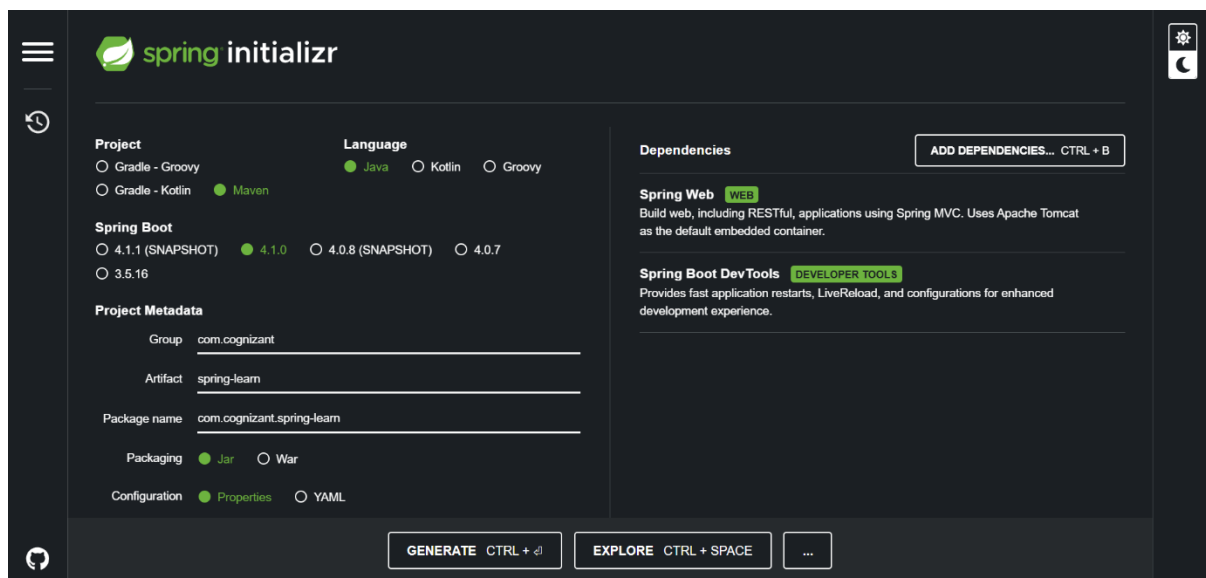


1.Hands on 1-Create a Spring Web Project using Maven

Creating Spring Boot Project



SpringLearnApplication.java

```
package com.cognizant.spring_learn;

import org.slf4j.Logger;

import org.slf4j.LoggerFactory;

import org.springframework.boot.SpringApplication;

import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class SpringLearnApplication {

    private static final Logger LOGGER =

        LoggerFactory.getLogger(SpringLearnApplication.class);
```

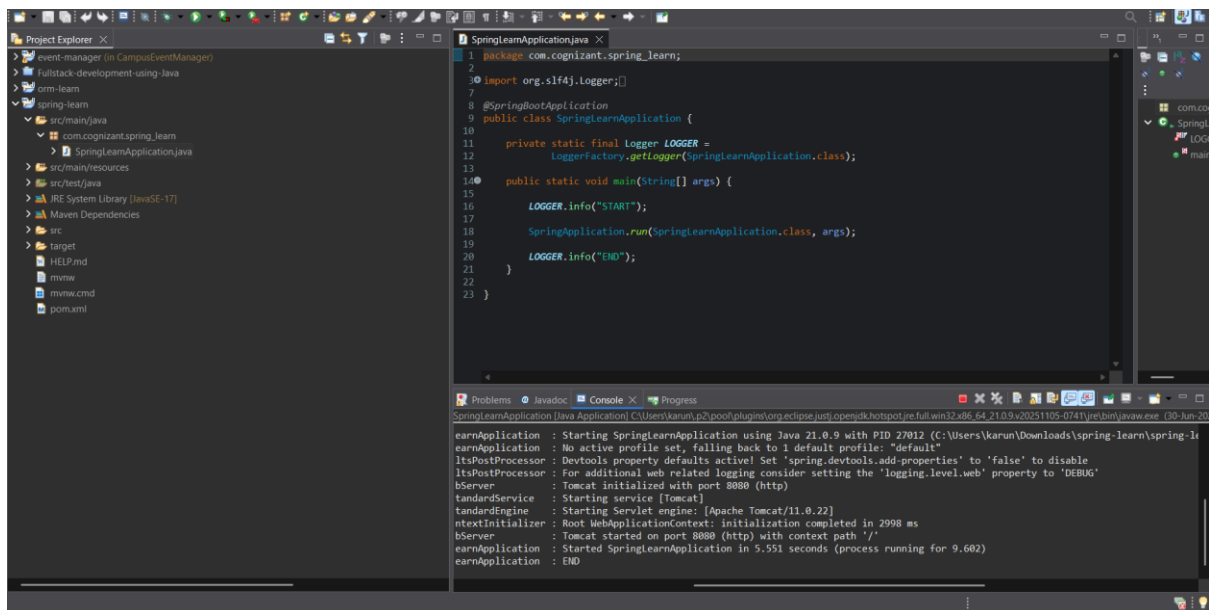
```

public static void main(String[] args) {
    LOGGER.info("START");

    SpringApplication.run(SpringLearnApplication.class, args);

    LOGGER.info("END");
}
}

```



1.Hands on 4-Spring Core Load Country from Spring Configuration XML

Country.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="
```

```
http://www.springframework.org/schema/beans
```

<https://www.springframework.org/schema/beans/spring-beans.xsd>">

<bean id="country"

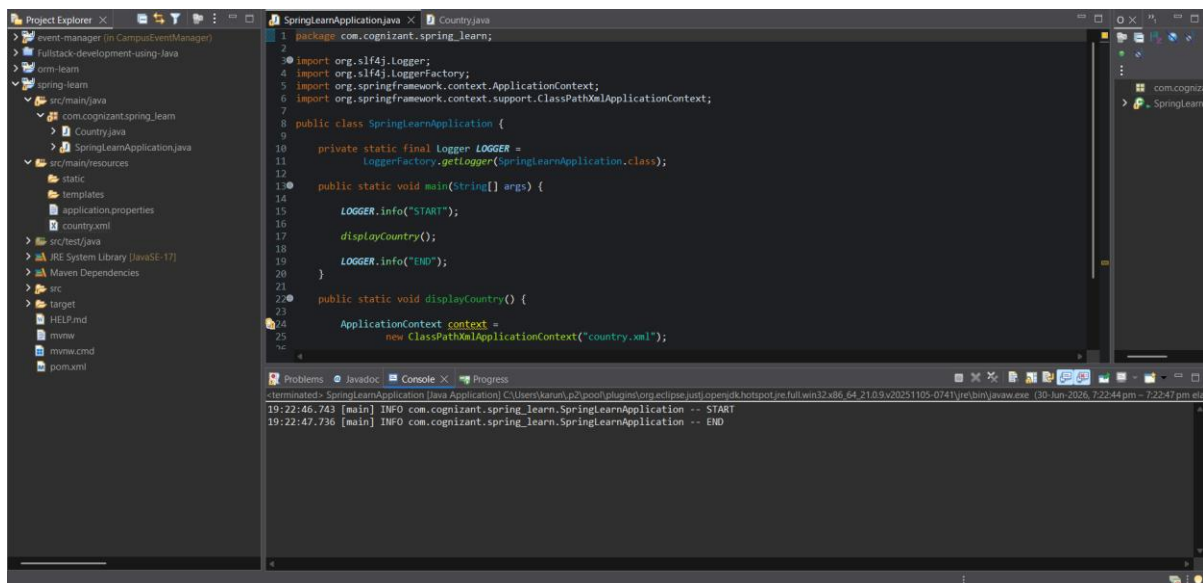
class="com.cognizant.spring_learn.Country">

<property name="code" value="IN"/>

<property name="name" value="India"/>

</bean>

</beans>



2.Hands on 2-Hello World RESTful Web Service

HelloController.java

```
package com.cognizant.spring_learn.controller;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController

public class HelloController {

    private static final Logger LOGGER =

        LoggerFactory.getLogger(HelloController.class);

    @GetMapping("/hello")

    public String sayHello() {

        LOGGER.info("START");

        String message = "Hello World!!!";

        LOGGER.info("END");

        return message;

    }

}

```

The screenshot shows an IDE with the following components:

- Project Explorer:** Shows the project structure with folders like `event-manager`, `Fullstack-development-using-java`, `orm-learn`, `spring-learn`, `src/main/java`, `src/main/resources`, `src/test/java`, `Maven Dependencies`, `src`, `target`, `HELP.md`, `mvnw`, `mvnw.cmd`, and `pom.xml`.
- Editor:** Displays the `HelloController.java` file with the following code:


```

1 package com.cognizant.spring_learn.controller;
2
3 import org.slf4j.Logger;
4
5 @RestController
6 public class HelloController {
7
8     private static final Logger LOGGER =
9         LoggerFactory.getLogger(HelloController.class);
10
11     @GetMapping("/hello")
12     public String sayHello() {
13
14         LOGGER.info("START");
15
16         String message = "Hello World!!!";
17
18         LOGGER.info("END");
19
20         return message;
21     }
22 }

```
- Console:** Shows the execution logs for the `SpringLearnApplication` and `SpringLearnApplication` classes. The logs indicate that the application started successfully on port 8083 and that the `sayHello()` method was executed.


```

2026-06-30T22:17:58.011+05:30 INFO 32236 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-06-30T22:17:58.013+05:30 INFO 32236 --- [spring-learn] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.22]
2026-06-30T22:17:58.146+05:30 INFO 32236 --- [spring-learn] [ restartedMain] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed
2026-06-30T22:17:59.148+05:30 INFO 32236 --- [spring-learn] [ restartedMain] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8083 (http) with context path
2026-06-30T22:17:59.158+05:30 INFO 32236 --- [spring-learn] [ restartedMain] c.c.spring_learn.SpringLearnApplication : Started SpringLearnApplication in 4.967 seconds (pro
2026-06-30T22:17:59.168+05:30 INFO 32236 --- [spring-learn] [ restartedMain] c.c.spring_learn.SpringLearnApplication : END
2026-06-30T22:18:37.374+05:30 INFO 32236 --- [spring-learn] [nio-8083-exec-1] o.a.c.c.f.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherSer
2026-06-30T22:18:37.377+05:30 INFO 32236 --- [spring-learn] [nio-8083-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-06-30T22:18:37.461+05:30 INFO 32236 --- [spring-learn] [nio-8083-exec-1] c.c.s.controller.HelloController : Completed initialization in 3 ms
2026-06-30T22:18:37.462+05:30 INFO 32236 --- [spring-learn] [nio-8083-exec-1] c.c.s.controller.HelloController : START
2026-06-30T22:18:37.462+05:30 INFO 32236 --- [spring-learn] [nio-8083-exec-1] c.c.s.controller.HelloController : END

```

Execution Image



2.Hands On 3-REST Country Web Service

CountryController.java

```
package com.cognizant.spring_learn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import
org.springframework.context.support.ClassPathXmlApplicationConte
xt;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.spring_learn.Country;

@RestController
```

```

public class CountryController {

    private static final Logger LOGGER =

        LoggerFactory.getLogger(CountryController.class);

    @RequestMapping("/country")

    public Country getCountryIndia() {

        LOGGER.info("START");

        ApplicationContext context =

            new ClassPathXmlApplicationContext("country.xml");

        Country country =

            context.getBean("country", Country.class);

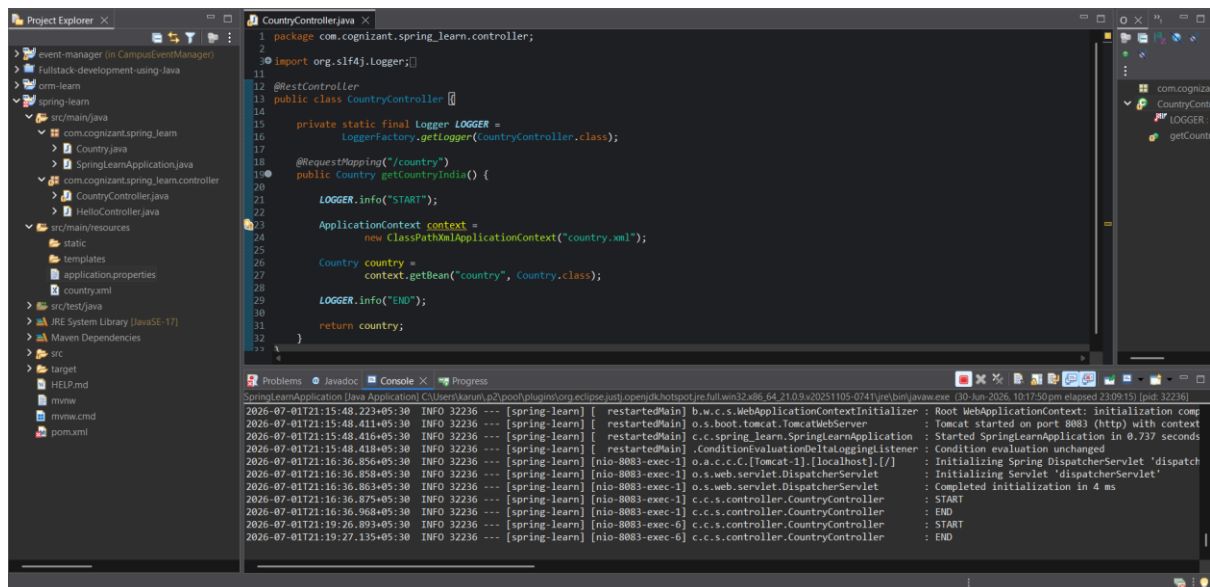
        LOGGER.info("END");

        return country;

    }

}

```



Execution Image

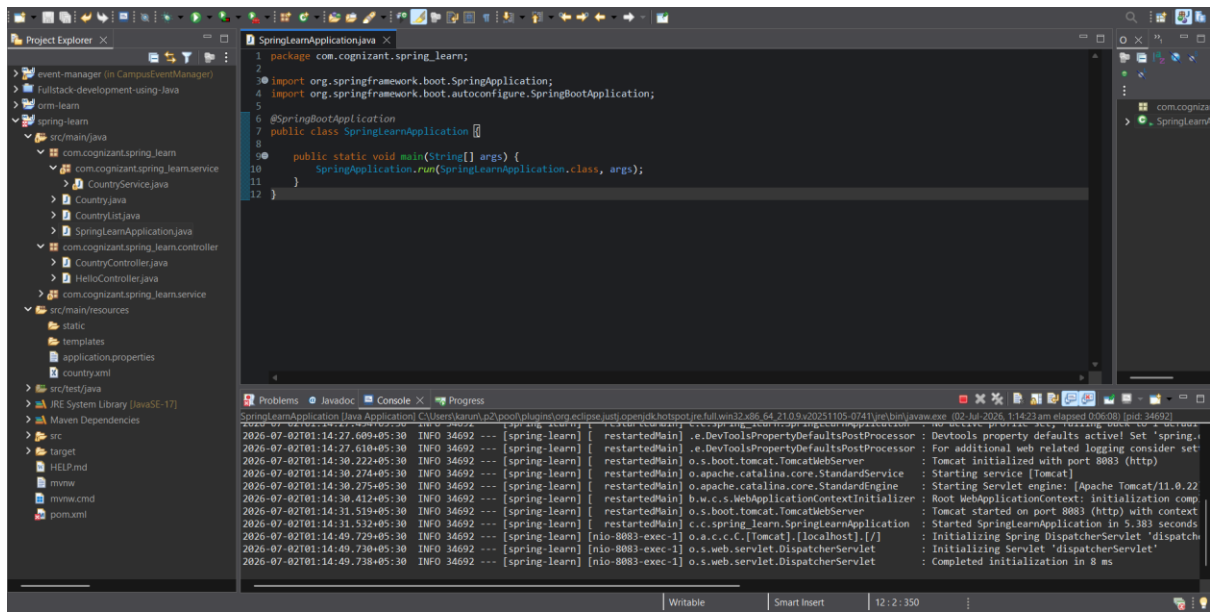
```
{  
  "code": "IN",  
  "name": "India"  
}
```

2.Hands On 5-REST Get country based on country code

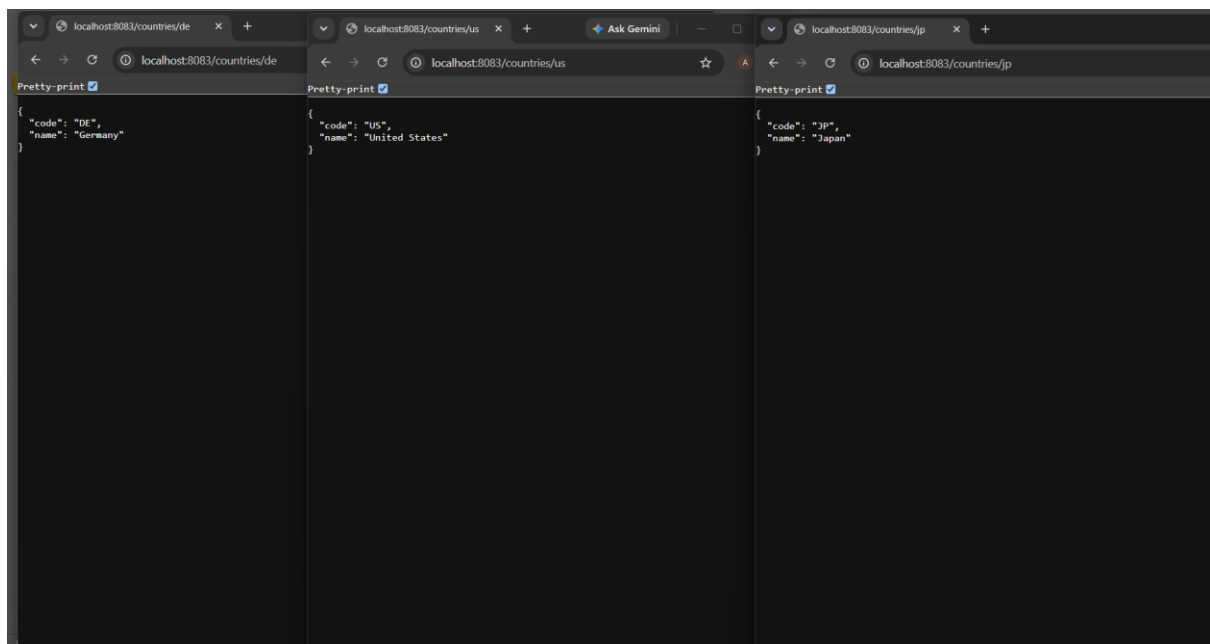
SpringLearnApplication.java

```
package com.cognizant.spring_learn;  
  
import org.springframework.boot.SpringApplication;  
  
import  
org.springframework.boot.autoconfigure.SpringBootApplication;  
  
@SpringBootApplication  
  
public class SpringLearnApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(SpringLearnApplication.class, args);  
    }  
}
```

}



Execution Image



JWT Hands On-Create authentication service that returns JWT

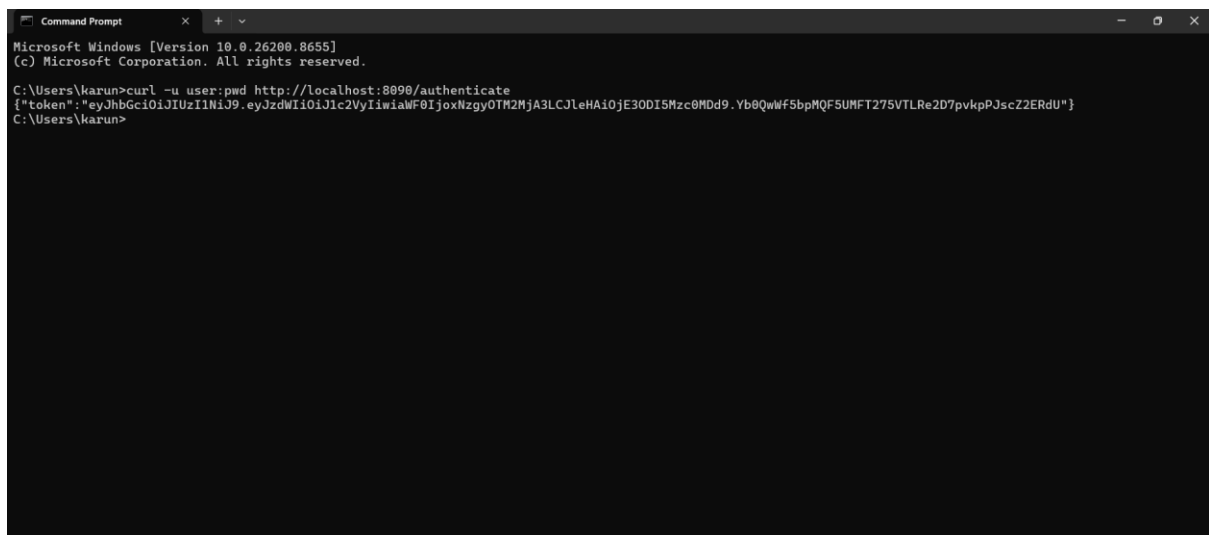
AuthenticationController.java

```
package com.cognizant.spring_learn.controller;

import java.nio.charset.StandardCharsets;
import java.util.Base64;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.spring_learn.model.AuthenticationResponse;
import com.cognizant.spring_learn.util.JwtUtil;

@RestController
public class AuthenticationController {

    @GetMapping("/authenticate")
    public AuthenticationResponse authenticate(
        @RequestHeader("Authorization") String authHeader) {
        String encoded =
            authHeader.substring("Basic ".length());
        byte[] decoded =
            Base64.getDecoder().decode(encoded);
        String credentials =
            new String(decoded, StandardCharsets.UTF_8);
        String username = credentials.split(":")[0];
```

$$\left. \begin{array}{l} \} \\ \} \end{array} \right\}$$


Server Port Output

